

**Федеральное государственное автономное образовательное учреждение
высшего образования «Санкт-Петербургский государственный
электротехнический университет «ЛЭТИ»
им. В.И. Ульянова (Ленина)»
кафедра ВТ**

ОТЧЕТ

по лабораторной работе №3

по дисциплине «Технологии Web-программирования»

**Тема: «ПОСТРОЕНИЕ web-ПРОЕКТА С ИСПОЛЬЗОВАНИЕМ
ТЕХНОЛОГИИ СЕРВЛЕТОВ»**

Студенты гр. 4314 _____

Селикаев Р. К.

Преподаватель _____

Павловский М. Г.

Санкт-Петербург

2026

Цель работы: знакомство с технологией построения web-приложений на основе сервлетов.

Порядок выполнения работы:

1. Выбрать предметную область для лабораторной работы и разработать для нее web-приложение (например, можно взять предметную область из курсового проекта по объектно-ориентированному программированию на языке Java).
2. Создать Maven-проект для web-приложения.
3. Разработать HTML-страницу и сервлет для web-приложения. Сервлет должен принимать параметры инициализации и один или более динамических параметров.
4. По команде Maven install создать war-архив и разместить его на сервере Apache Tomcat.
5. Запустить web-сервер и обратиться к сервлету из браузера, проверить правильность отображения HTML-страницы и работу сервлета.
6. Сгенерировать документацию для сервлета с помощью команды Maven build и просмотреть ее в браузере.

Настройка среды и разработка сервлета

В среде Eclipse создан Maven проект WebApp1 с использованием архетипа maven-archetype-webapp (версия 1.4). File → New → Project → Maven → Maven Project. Параметры проекта: 1)Group Id: org.edu; 2)Artifact Id: WebApp1; 3) Version: 0.0.1-SNAPSHOT; 4)Package: org.edu.webapp. Проект получил стандартную структуру web-приложения. Структура созданного проекта показана на рисунке 1.

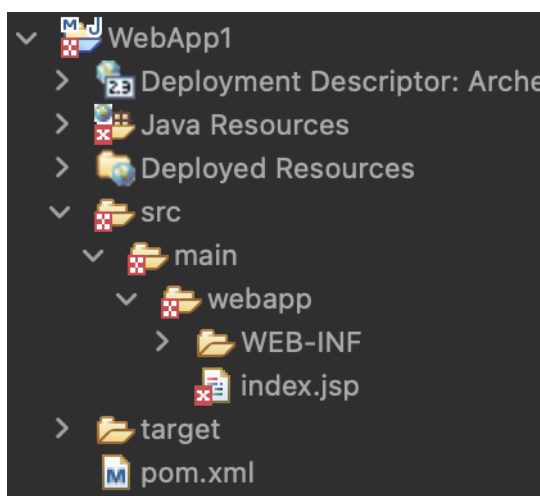


Рис. 1 – Структура Maven проекта WebApp1 для разработки сервлета

Для дальнейшей разработки сервлета во вновь созданном проекте мы открыли файл `pom.xml` и добавили зависимость *jakarta.servlet-api* версии 5.0.0. Распечатку измененного фрагмента файла конфигурации можно увидеть на рисунке 2.

```
22 <dependencies>
23   <dependency>
24     <groupId>junit</groupId>
25     <artifactId>junit</artifactId>
26     <version>4.11</version>
27     <scope>test</scope>
28   </dependency>
29   <dependency>
30     <groupId>jakarta.servlet</groupId>
31     <artifactId>jakarta.servlet-api</artifactId>
32     <version>5.0.0</version>
33     <scope>provided</scope>
34   </dependency>
35 </dependencies>
```

Рис. 2 – Измененный фрагмент файла конфигурации `pom.xml`

Вслед за этим из папки `webapp` нашего Maven проекта необходимо удалить файл `index.jsp`, что мы и сделали сразу после изменения файла конфигурации. Подтверждение удаления данного файла отображено на рисунке 3.

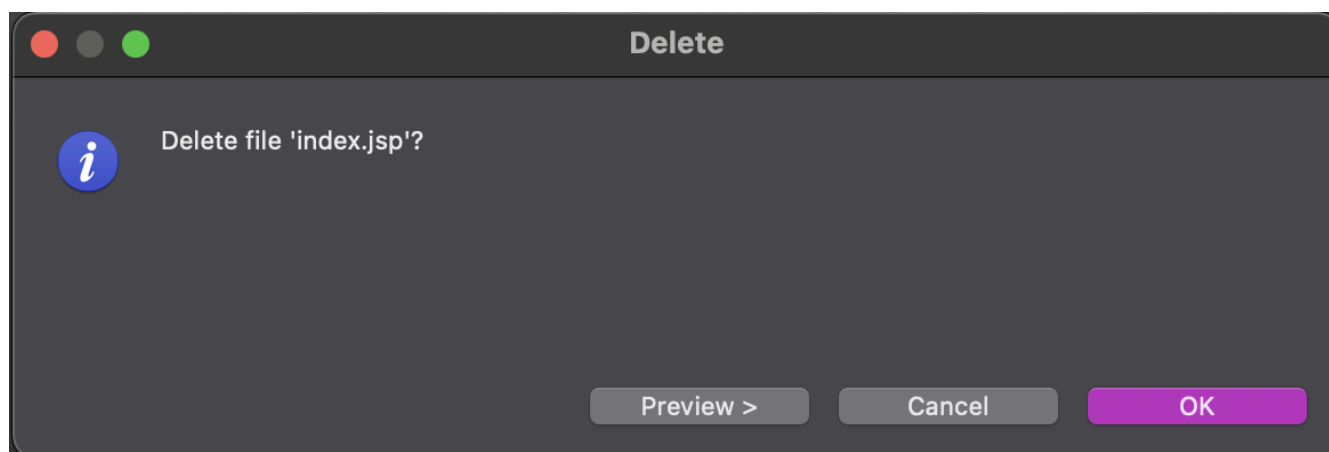


Рис. 3 – Удаление файла `index.jsp` из папки `webapp` проекта *WebApp1*

Для разработки сервлета создан класс *ComicsList* в пакете *lab.web*, расширяющий *jakarta.servlet.http.HttpServlet*. Сервлет принимает динамический параметр *name* (имя читателя) и выводит *HTML*-страницу со списком комиксов. После разработки сервлета и его загрузки в проект в директории `src/main/java/` появляется папка *lab.web*, содержащая файл *ComicsList.java*. Результат разработки сервлета и его добавление в проект представлен на рисунке 4.

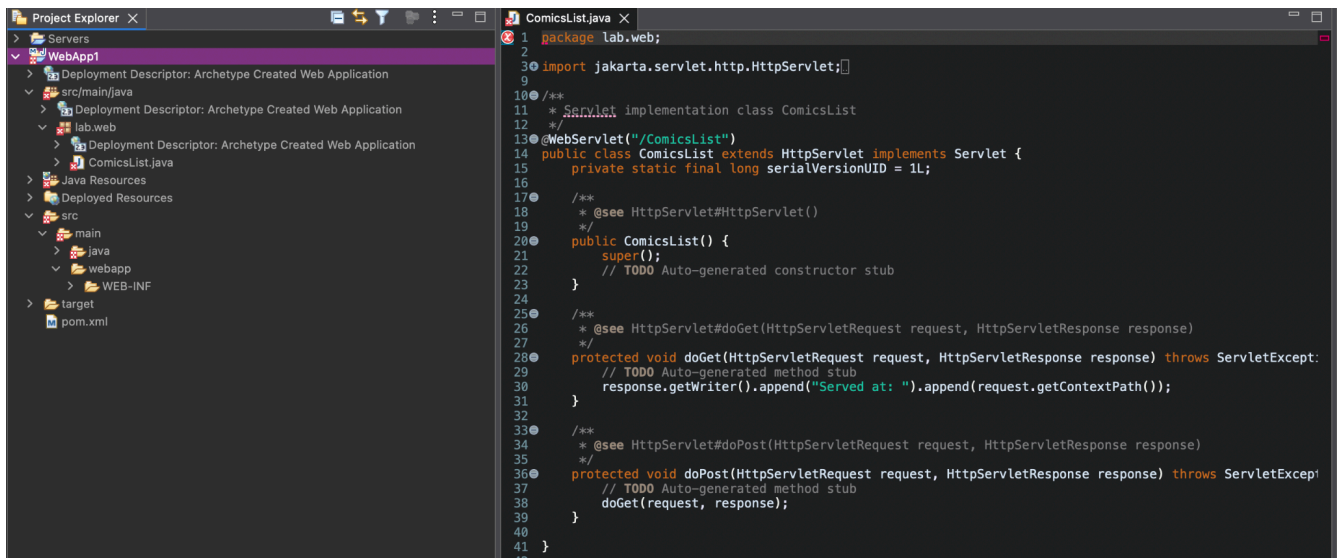


Рис. 4 – Разработка сервлета ComicsList.java

Добавление сервлета в проект нашего web-приложения автоматически меняет структуру другого файла конфигурации – web.xml. Туда после разработки ComicsList.java добавляются две секции: объявление сервлета и отображение сервлета. Системные изменения файла web.xml можно увидеть на рисунке 5.

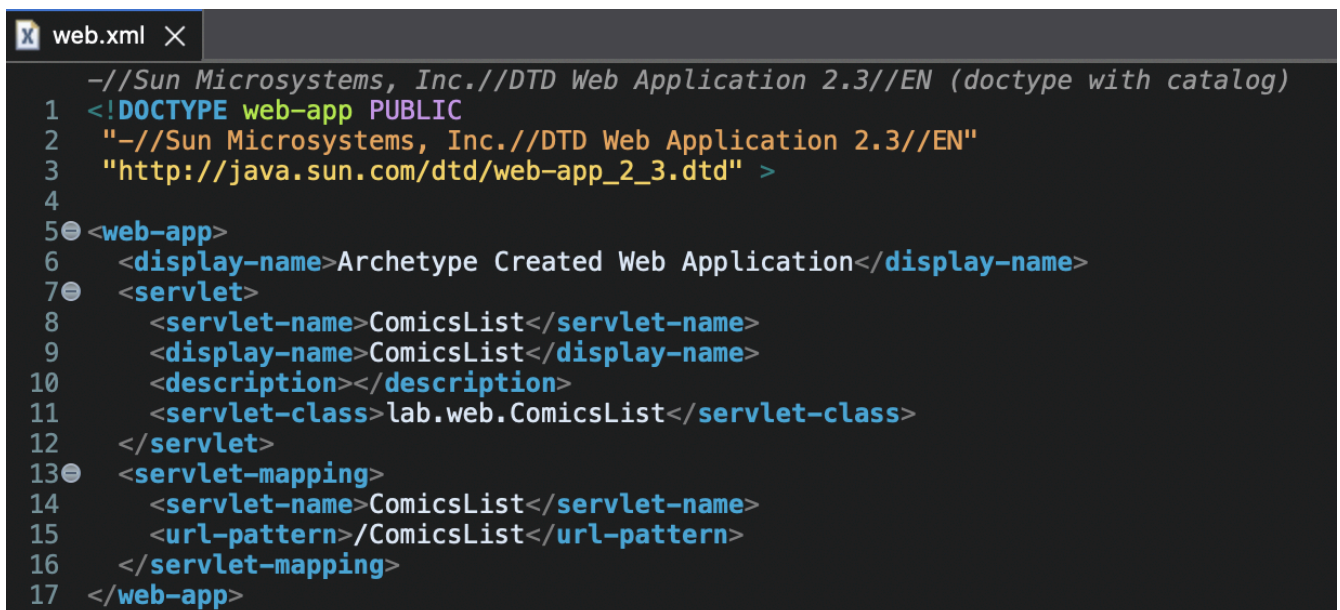


Рис. 5 – Измененный после регистрации сервлета файл конфигурации web.xml

После создания сервлета мы заменяем системный код файла ComicsList.java на код осуществляющий создание HTML-страницы со списком прочитанных комиксов. Новый код сервлета показан на рисунке 6.

```

package lab.web;
import jakarta.servlet.ServletException;

/**
 * Servlet implementation class ComicsList.
 */
public class ComicsList extends HttpServlet {
    private static final long serialVersionUID = 1L;

    /**
     * @see HttpServlet#HttpServlet()
     */
    public ComicsList() {
        super();
    }

    /**
     * Обрабатывает запросы GET и POST.
     *
     * @param request HTTP-запрос
     * @param response HTTP-ответ
     * @throws ServletException
     * @throws IOException
     */
    protected void processRequest(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        request.setCharacterEncoding("utf-8");
        String name = request.getParameter("name");

        response.setContentType("text/html;charset=UTF-8");
        PrintWriter out = response.getWriter();
        try {
            out.println("<html>");
            out.println("<head><title>Список комиксов</title></head>");
            out.println("<body>");
            out.println("<h1>Список комиксов читателя "
                + (name != null ? name : "без имени") + "</h1>");
            out.println("<table border='1'>");
            out.println("<tr><td><b>Автор комикса</b></td>"
                + "<td><b>Название комикса</b></td>"
                + "<td><b>Прочитал</b></td></tr>");
            out.println("<tr><td>Джонатан Хикман</td><td>Секретные Войны</td><td>Да</td></tr>");
            out.println("<tr><td>Брайан Бендис</td><td>День М</td><td>Да</td></tr>");
            out.println("<tr><td>Донни Кейтс</td><td>Веном</td><td>Нет</td></tr>");
            out.println("</table>");
            out.println("</body>");
            out.println("</html>");
        } finally {
            out.close();
        }
    }

    /**
     * @see HttpServlet#doGet(HttpServletRequest request, HttpServletResponse response)
     */
    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        processRequest(request, response);
    }

    /**
     * @see HttpServlet#doPost(HttpServletRequest request, HttpServletResponse response)
     */
    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        processRequest(request, response);
    }
}

```

Рис. 6 – Новый код сервлета *ComicsList.java*

Запуск сервлета

Перед запуском сервлета необходимо выполнить команду Maven install. По этой команде в каталоге target будет создан файл *WebApp1.war* (см. рис. 7). Этот файл надо скопировать в каталог webapps и запустить сервер Tomcat. Для запуска сервлета на локальном компьютере при использовании установленного порта (в нашем случае 8081) необходимо в браузере задать адрес, в котором кроме этих данных надо указать имя каталога *WebApp1* и имя сервлета. Для созданного в процессе выполнения лабораторной сервлета адрес будет следующим:

<http://localhost:8081/WebApp1/ComicsList>

Если в запросе требуется передать дополнительные динамические параметры, то после адреса надо поставить знак «?» и задать имя параметра и его значение, например: <http://localhost:8080/WebApp1/BooksList?name=Иванова И.И.>

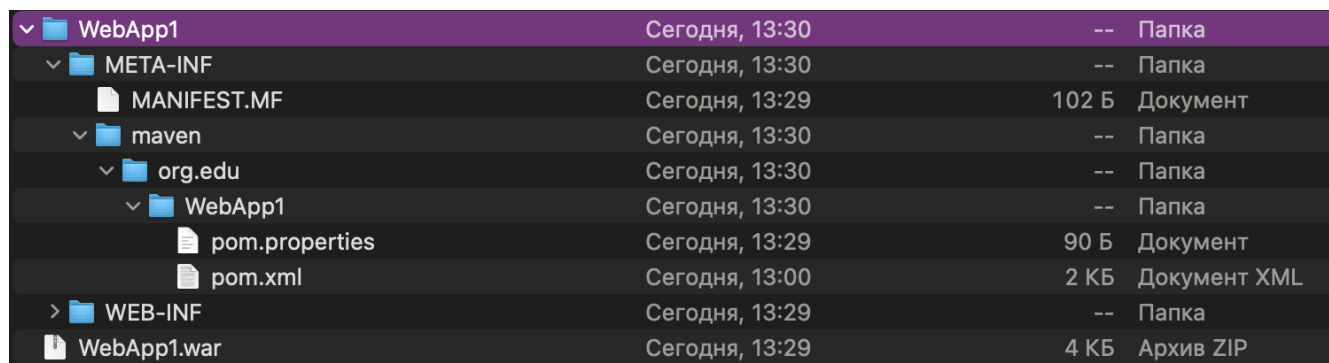


Рис. 7 – Создание файла *WebApp1.war* в папке *target* после выполнения команды *Maven install* и отображение его структуры и состава

После того как файл создан и скопирован в папку *webapps* мы можем запустить web-сервер и проверить корректность работы приложения. В качестве тестирования были открыты через браузер два адреса с параметром *name* и без него. Результаты работы web-приложения можно увидеть на рисунках 8-9.

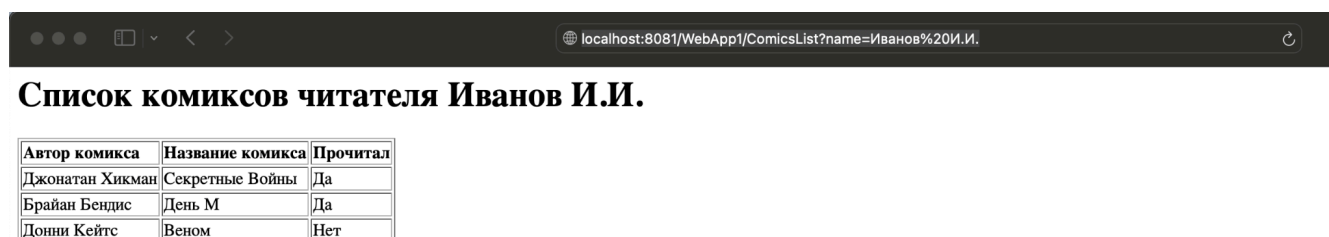


Рис. 8 – Открытие web-приложения с параметром *name*

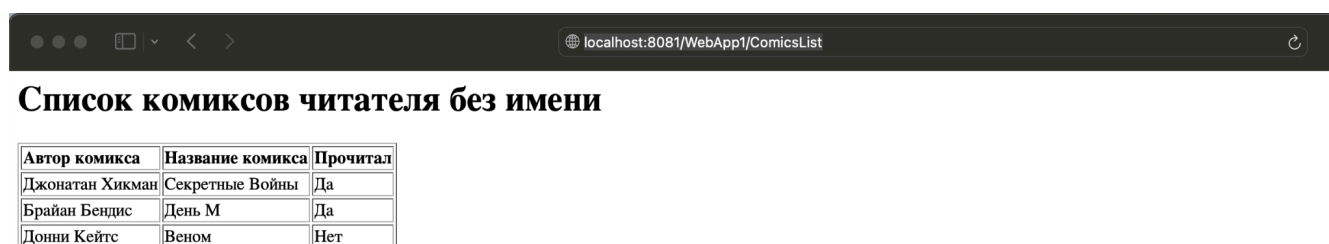


Рис. 9 – Открытие web-приложения без параметра

Документация проекта

Для документации проекта использовалась команда терминала *mvn javadoc:javadoc* в среде нашего Maven проекта. В итоге мы получили файл документации в директории *target/reports/apidocs/lab/web*. Содержание файла документации отображено на рисунке 10.

Class ComicsList

[java.lang.Object](#)
[jakarta.servlet.GenericServlet](#)
[jakarta.servlet.http.HttpServlet](#)
[lab.web.ComicsList](#)

All Implemented Interfaces:
[jakarta.servlet.Servlet](#), [jakarta.servlet.ServletConfig](#), [Serializable](#)

public class ComicsList extends [jakarta.servlet.http.HttpServlet](#)
Servlet implementation class ComicsList.

See Also:

- [Serialized Form](#)

• Constructor Summary

Constructors
Constructor
Description
[ComicsList\(\)](#)

• Method Summary

All MethodsInstance MethodsConcrete Methods

Modifier and Type
Method
Description
protected void
[doGet\(\)](#)([jakarta.servlet.http.HttpServletRequest](#) request, [jakarta.servlet.http.HttpServletResponse](#) response)

protected void
[doPost\(\)](#)([jakarta.servlet.http.HttpServletRequest](#) request, [jakarta.servlet.http.HttpServletResponse](#) response)

protected void
[processRequest\(\)](#)([jakarta.servlet.http.HttpServletRequest](#) request, [jakarta.servlet.http.HttpServletResponse](#) response)
Обработывает запросы GET и POST.

Methods inherited from class [jakarta.servlet.http.HttpServlet](#)

doDelete, doHead, doOptions, doPut, doTrace, getLastModified, service, service

Methods inherited from class [jakarta.servlet.GenericServlet](#)

destroy, getInitParameter, getInitParameterNames, getServletConfig, getServletContext, getServletInfo, getServletName, init, init, log, log

Methods inherited from class [java.lang.Object](#)

[clone](#), [equals](#), [finalize](#), [getClass](#), [hashCode](#), [notify](#), [notifyAll](#), [toString](#), [wait](#), [wait](#), [wait](#)

• Constructor Details

◦ ComicsList

public ComicsList()
See Also:

- [HttpServlet\(\)](#)

• Method Details

◦ processRequest

protected void processRequest([jakarta.servlet.http.HttpServletRequest](#) request, [jakarta.servlet.http.HttpServletResponse](#) response) throws [jakarta.servlet.ServletException](#), [IOException](#)
Обработывает запросы GET и POST.
Parameters:
request - HTTP-запрос
response - HTTP-ответ
Throws:
[jakarta.servlet.ServletException](#)
[IOException](#)

◦ doGet

protected void doGet([jakarta.servlet.http.HttpServletRequest](#) request, [jakarta.servlet.http.HttpServletResponse](#) response) throws [jakarta.servlet.ServletException](#), [IOException](#)
Overrides:
doGet in class [jakarta.servlet.http.HttpServlet](#)
Throws:
[jakarta.servlet.ServletException](#)
[IOException](#)
See Also:

- [HttpServlet.doGet\(\)](#)([HttpServletRequest](#) request, [HttpServletResponse](#) response)

◦ doPost

protected void doPost([jakarta.servlet.http.HttpServletRequest](#) request, [jakarta.servlet.http.HttpServletResponse](#) response) throws [jakarta.servlet.ServletException](#), [IOException](#)
Overrides:
doPost in class [jakarta.servlet.http.HttpServlet](#)
Throws:
[jakarta.servlet.ServletException](#)
[IOException](#)
See Also:

- [HttpServlet.doPost\(\)](#)([HttpServletRequest](#) request, [HttpServletResponse](#) response)

Рис. 10 – Документация Maven проекта лабораторной работы

7

Вывод по лабораторной работе: в ходе лабораторной работы изучена технология *Java Servlet* – механизм построения серверной части web-приложений. Разработан сервлет *ComicsList*, обрабатывающий *HTTP*-запросы *GET* и *POST*, принимающий динамический параметр *name* и формирующий *HTML*-страницу со списком комиксов. Освоена работа с *Maven* для сборки *WAR*-архива, содержащего сервлет и дескриптор развертывания *web.xml*. *WAR*-архив развернут на web-сервере *Apache Tomcat*, приложение проверено в браузере. Сгенерирована *Javadoc* документация, описывающая класс *ComicsList*, конструктор и все три метода (*processRequest*, *doGet*, *doPost*).

Приложение

Ссылка на репозиторий с проектом лабораторной работы представлена ниже:

<https://github.com/romaselikaev/web-tech/tree/main/lab3>